

Project 1-3: Implementing DML

How-to

- **Python 3.12** should be installed in the system.
- You can run `make` to automatically setup the virtual environment and run the DBMS.
- You can run `make test` to automatically setup the virtual environment and run the tests.
- Or, you can manually setup the environment as `requirements.txt` is provided.

Implementation

`grammar.lark`

- Added rules for `JOIN ... ON`, `ORDER BY ... [ASC|DESC]`, and `GROUP BY` so that they hang off `table_expression` together with the existing `where_clause`, `limit_clause`, and `offset_clause`.
- Added `aggregate_func` (`MAX` | `MIN` | `SUM`) wrapping a `column_ref` inside `selected_column`.
- Promoted comparison operators (`=`, `!=`, `<`, `<=`, `>`, `>=`) to named tokens (`EQ`, `NE`, ...) so the transformer can read them — anonymous inline string terminals are filtered out of the tree by Lark.
- Extracted `column_ref` as a reusable rule so `SELECT`, `JOIN ON`, `WHERE`, `ORDER BY`, and `GROUP BY` all share the same `[table. ]col` shape.

`run.py`

Result `dataclass`

- Carries an output text and a `prompt: bool` flag. `prompt=False` is used for tabular output (`EXPLAIN`, `SELECT`, `SHOW TABLES`) so that the `DB_2020-17316>` prompt is omitted before the table body. Replaces the earlier heuristic of inspecting `"\n"` in `res` for layout decisions.

Database `class`

- Unchanged from Project 1-2, plus a small `set_rows()` helper used by `DELETE` to overwrite the surviving rows in one shot.

`SQLTransformer class`

- DDL handlers from 1-2 are preserved as-is.
- `value()` now returns a tagged `{"v": ..., "t": "int"|"char"|"date"|"null"}` dict so `INSERT` can validate per-value types against the column schema without re-parsing.
- `WHERE` / `JOIN ON` / `ORDER BY` / `GROUP BY` rules build small AST nodes shaped as `{"k": ..., ...}` dicts (matching the convention already used by `column_definition` and friends). The `boolean_expr`, `boolean_term`, `boolean_factor` chain is folded into nested `{"k": "and"|"or"|"not", ...}` trees.

Helpers (module-level)

- `_resolve_column(ref, scope, clause)` walks the FROM scope, raising `_QueryError("TABLE_NOT_SPECIFIED" | "COLUMN_NOT_EXIST" | "AMBIGUOUS", clause=...)`. It is shared by every clause that resolves columns.
- `_bind_expr()` walks the WHERE AST once, replaces each column reference with its resolved schema entry, and validates comparison-operand type compatibility (raising `INCOMPARABLE`).
- `_eval_expr()` is a three-valued (`True` | `False` | `None`) evaluator over the bound AST.
- `_aggregate(fn, vals)` implements `MAX` / `MIN` / `SUM` with the spec's null/empty rules (`SUM` returns `0` on non-int columns and all-null input; `MAX`/`MIN` return `NULL`).
- `_msg_for(e)` translates `_QueryError` codes back into the exact message strings from `2026-1_1-3_Messages.pdf`. Centralising this keeps clause-specific text (e.g. `where`, `order by`, `group by`, `join`) in one place.

Query handlers

- **INSERT.** Checks table existence, column-list validity, length match, per-value type compatibility (with `NULL` accepted as any type), then enforces the `NOT NULL` constraint, then truncates oversize `char` values, then persists.
- **DELETE.** Builds a single-table scope, binds the WHERE expression, filters rows, and runs an FK integrity pass: for every other table, every FK pointing here, compare the child row's FK values against the parent row's referenced columns. If any matched row is referenced, no row is deleted and `DeleteReferentialIntegrityPassed` is emitted with the matched count (per spec, exclusive with `DeleteResult`).
- **SELECT.** Resolves the FROM list (mix of base entries and JOINS), binds JOIN conditions / WHERE / ORDER BY / GROUP BY columns against that scope, validates SELECT-list resolution (failures become `SelectColumnResolveError`), then executes:
 1. Cross-product over all FROM tables.
 2. Filter by JOIN equality conditions (drop nulls per SQL semantics).
 3. Filter by WHERE (rows kept only when the evaluator returns `True`).
 4. If GROUP BY (or any aggregate in the select-list) is present, hash rows into groups and reduce each group's columns through `_aggregate()`. The first-seen group order is preserved before ORDER BY runs.
 5. ORDER BY: split the rows into null vs non-null on the sort key, sort the non-null partition, and append the null partition at the end for ASC / start for DESC (MySQL default). Using a partition avoids `None < None` `TypeError` on mixed sort keys.
 6. Apply OFFSET then LIMIT (both validated as non-negative).

`test.sh`

- 18 sections, 83 tests in total, covering: all DDL from 1-2, INSERT happy + 4 error types, DELETE with WHERE and FK integrity, SELECT projection / WHERE (= , IS NULL , AND/OR), JOIN (2 and 3 tables, type mismatch), ORDER BY ASC/DESC, LIMIT/OFFSET, every clause-specific error message, and the GROUP BY bonus (MAX/MIN/SUM, single-group aggregate, SelectColumnNotGrouped).
- The `normalize` helper collapses runs of dashes to `---` and trims trailing whitespace so the tests don't break when column widths change.

Additional Implementation

- **Single-group aggregates without GROUP BY.** Spec only defines aggregates with `GROUP BY`. We follow MySQL: an aggregate in the select-list without `GROUP BY` reduces the whole filtered row set into one group (e.g. `SELECT max(v) FROM t` yields one row).
- **Null-ordering convention for ORDER BY.** Spec is silent on null placement. We use MySQL's default: nulls last for `ASC`, nulls first for `DESC`.
- **Aggregate column header.** Headers follow the source: `agg(table.col)` if the user qualified the column, `agg(col)` otherwise; an explicit `AS alias` overrides both.
- **SELECT *** with multiple tables in FROM. Spec only shows `SELECT *` on a single table. With multiple tables we prefix every header with the table alias (`t1.col`, `t2.col`) to keep the output unambiguous when both tables share column names.
- No additional error types were defined beyond those listed in `2026-1_1-3_Messages.pdf`.

My Impression

- Once column resolution and operand type-checking were pulled into `_resolve_column` and `_bind_expr`, the per-clause handlers stayed small.
- Anonymous string terminals in Lark are dropped from the parse tree by default. The grammar quietly type-checked but every `=` comparison crashed at runtime until I named the operator tokens.